PART 3

EXPERT LEVEL QUES-TIONS

CHAPTER 15

DESIGN PATTERNS, PRINCIPLES, ARCHITEC-TURES

111. What design patterns are commonly used in Android apps?

Design patterns and principles are huge topics, i can't explain them one by one because it may take days even weeks. But i can briefly explain used design patterns in Android.

Builder
Builder pattern separates construction of a complex object from its representation. In Android, the Builder pattern appears when using objects like AlertDialog.Builder.

```
AlertDialog.Builder( this )
```

```
    .setTitle( "Mahatma Gandhi " )
    .setMessage( "Be the change you wanna see in
the world." )
    .setNegativeButton( "No thanks" , { dialogIn-
terface, i ->
        // "No thanks" button was clicked
    } )
    .setPositiveButton( "OK" , { dialogInterface, i ->
        // "OK" button was clicked
    } )
    .show()
```

Dependency Injection

Dependency injection has you provide any objects required when you instantiate a new object; the new object doesn't need to construct or customize the objects itself.

Dagger 2 is the most popular open-source dependency injection framework for Android and was developed in collaboration between Google and Square.

```
@Module
class AppModule {
  @Provides fun provideSharedPreferences(app:
Application): SharedPreferences {
    return app.getSharedPreferences( "prefs" ,
Context. MODE_PRIVATE )
  }
}

//-----------------------------------------------------
-------------------------------------

@Inject lateinit var sharedPreferences: Shared-
Preferences
```

Singleton

The Singleton Pattern specifies that only a single instance of a class should exist with a global point of access. **Application** class of Android is a good example of Singleton.

Adapter

Adapter Pattern matches interfaces of different classes. That is the most common pattern used in Android. Because we use it to bind RecyclerViews. We have adapter object for them.

```
class MyAdapter( private val data : List<Data>) :
RecyclerView.Adapter<MyViewHolder>() {
  override fun onCreateViewHolder(viewGroup:
ViewGroup, i: Int): MyViewHolder {
    val inflater = LayoutInflater.from(viewGroup.
context )
    val view = inflater.inflate(R.layout.row, view-
Group, false )
    return MyViewHolder(view)
  }
  override fun onBindViewHolder(viewHolder:
MyViewHolder, i: Int) {
    viewHolder.bind( data [i])
  }
  override fun getItemCount() = data . size
}
```

Facade

The Facade pattern provides a higher-level interface that makes a set of other interfaces easier to use. **Retrofit** helps us implement the Facade pattern; we create an interface to provide API data to client classes like so:

```
interface BooksApi {
  @GET( "books" )
  fun listBooks(): Call<List<Book>>
}
```

Command

Command pattern lets you issue requests without knowing the receiver. **EventBus** is a popular Android framework that supports this pattern in the following manner:

```
class MySpecificEvent { /* Additional fields if
needed */ }
eventBus.register( this )
fun onEvent(event: MySpecificEvent) {
```

```
  /* Do something */
}
eventBus.post(event)
```

Observer

Observer Pattern is a way of notifying change to a number of classes. The **RxAndroid** framework lets us implement this pattern throughout our app. A sample usage would be:

```
apiService.getData(someData)
.subscribeOn(Schedulers.io())
.observeOn(AndroidSchedulers.mainThread())
.subscribe ( /* an Observer */ )
```

MVVM (Model View ViewModel)

The MVVM pattern is trending upwards in popularity but is still a fairly recent addition to the pattern library. Google recently introduced this pattern as part of its **Architecture Components** library.

Important:

Design Patterns are very important for senior developers. You should be aware of widely used patterns in Android world. I have provided a good answer to this question. I recommend you to read the answer 10 times and repeat it to yourself. This way you are going to master it easily.

112. What is MVP?

The MVP pattern allows separating the presentation layer from the logic so that everything about how the UI works is agnostic from how we represent it on screen.

MVP is not an architecture by itself, it's only responsible for the presentation layer.

MVP only models the presentation layer, but the rest of layers will still require a good architecture if you want a flexible and scalable App.

An example of a complete architecture could be Clean Architecture, though there are many other options.

In MVP, we have four components: Activity/Fragment, View and Presenter interfaces and PresenterImpl class.

Our Activity/Fragment implements View interface. View interface contains methods related to events to occur on UI.

Presenter interface contains methods regarding to actions that UI may trigger. Presenter interface is implemented by PresenterImpl class.

That PresenterImpl class that implements Presenter interface. PresenterImpl keeps logic related to UI. It determines what will happen on screen.

And PresenterImpl keeps a reference to View, so that it can update Activity/Fragment whenever it needs.

A simple code would be as follows...

View Interface:

```
interface View{
   fun showLoading()
   fun hideLoading()
   fun onLoginSuccess(user: User)
   fun onLoginError(message: String)
}
```

Presenter Interface:

```
interface Presenter{
   fun login(userName:String, password:String)
}
```

PresenterImpl class:

```
class PresenterImpl( val view : View): Presenter{
   override fun login(userName: String, password:
String) {
      if (userNameAndPasswordCoorect(userName,
password)){
         view .hideLoading()
         view .onLoginSuccess(user)
      } else {
         view .hideLoading()
         view .onLoginError( "Incorrect username
or password" )
      }
   }
}
```

And Activity or Fragment:

```
class MVPActivity: AppCompatActivity(), View{
   private val presenter = PresenterImpl( this )
   override fun onLoginSuccess(user: User) {
   }
   override fun onLoginError(message: String) {
   }
}
```

Separating interface from logic in Android is not easy, but the MVP pattern makes it easier to prevent our activities end up degrading into very coupled classes consisting of hundreds or even thousands of lines. In large Apps, it is essential to organize our code well. Otherwise, it becomes impossible to maintain and extend.

Nowadays, there are other alternatives like MVVM, let's wait and see what happens.

113. What is MVVM?

MVVM stands for **Model-View-ViewModel** .
MVVM has three components:

The View: That informs the ViewModel about the user's actions.
The ViewModel: Exposes streams of data relevant to the View.
The Model: Abstracts the data source. The ViewModel works with the Model to get and save the data.

Basically, MVVM was proposed by Google with declaration of Architecture Components. Android Studio now provides a template for Fragment+ViewModel.

We are supposed to create a ViewModel for every screen. In the ViewModel we keep all screen related data in LiveData objects. Whenever this data is changed, it automatically triggers our UI components such as Fragments or Activities.

Let's visualize it with a simple diagram:

Every view is connected to a ViewModel. ViewModel keeps screen related logic and all the data that UI requires. It determines what will happen on the UI section. And ViewModel uses data classes while executing its job.

When compared to MVP, MVVM looks quite similar. The only difference is that, ViewModel replaces Presenter class of MVP.

Basic advantage is that, MVP may brings some lifecycle issues on runtime. Because we have to give a Context object and a View object to Presenter to do its job. But when Presenter tries to update UI, context or view object may no longer be alive. That causes some lifecycle problems potentially.

On the other hand, MVVM does not require to get an instance of view or context. Because this is handled by Architecture Components.

Let's explain MVVM with a sample code.

Here is a simple ViewModel. It keep screen related data in LiveData. LiveData is an observable data holder. Whenever data is changed, it notifies observers.

```
class MyViewModel : ViewModel() {
  private lateinit var users : MutableLive-
Data<List<User>>
  public fun getUsers(): LiveData<List<User>> {
    if ( users == null ) {
      users = MutableLiveData<List<User>>();
      loadUsers();
    }
    return users ;
  }
  private fun loadUsers() {
    // Do an asynchronous operation to fetch users.
  }
}
```

User object is our Model class but for brevity i am not providing source code for it. It is quite simple to imagine what are the fields of a simple User class.

Now we have ViewModel and its data in LiveData. Now we need observers, i mean screens: Fragments or Activities.

Here is a sample Activity that uses ViewModel and observes data inside it.

```
class MyActivity: AppCompatActivity() {
  public override fun onCreate( savedInstanceS-
tate: Bundle) {
    // Create a ViewModel the first time the system
calls an activity's onCreate() method.
    // Re-created activities receive the same MyView-
Model instance created by the first activity.
    val model =
ViewModelProviders.of( this ).get(MyViewModel.
class );
    model.getUsers().observe( this , users -> {
      // update UI
    } );
  }
}
```

Architecture Components provides us `ViewModelProviders` class to get an instance of ViewModel. We just provide class name of ViewModel to `ViewModelProviders` .

Then as you can see, we observe to users in ViewModel with line of `model.getUsers().observe()` . Whenever user data is changed, this callback is triggered immediately. That is how we keep screen related data in ViewModel and observe it via Observer Pattern. This pattern is simply called as Hollywood Principle.

To sum up, View is an activity or Fragment that observes data. ViewModel is a special class that holds UI data and logic. Model is simple data classes that we know.

MVVM is highly trending, i am aware of that and i am a big fan of it.

114. What are the common layers of responsibility in a well structured Android app?

In a well structured Android app, common layers would be **Presentation** , **Data** and **Domain** layers.

This structure is commonly used by Google's sample projects on Github.

Presentation is the layer that we keep everything related to UI. Activites, Fragments, ViewModels, custom UI components etc.

Data is the layer that fetches data. It may fetch from different sources such as network, web service, local db, file storage, SharedPrefs or cache or anywhere else. It doesn't matter. The only thing matters is that data layer is the only where that fetches data. It's the single source of truth all the app data.

Data layer may contain Retrofit related staff, some model classes with Retrofit annotations and some mapper classes if needed.

Domain is the layer that keeps business logic of our app. It has some classes called Use Case. Every use case contains an algorithm of a user scenario in the app. For instance, "Fetch shops in

coordinates of X" is a scenario and it has a corresponding use case class in domain section.

Domain section is not allowed to reach other layers: Presentation or Data. It can't reference a class or an object in other layers. It's also not allowed to use any third party libraries. It must be pure Kotlin or Java. Main idea behind this is it should be testable.

Domain data should be tested without any mocks or dependencies. That is why we try to reduce dependencies of this layer as much as possible.

Actually this architecture is named as Clean Architecture and proposed by Uncle Bob. His real name is Robert C. Martin, a famous guy in software world. Recently, Google also recommends this architecture or its derivatives.

When it comes to the architecture, i try to remember things that i have learned from famous video of Uncle Bob: Architecture, The Lost Years. Basically he says that we have wasted many years to find the perfect architecture but only a few us were lucky enough to find out that there is no perfect architecture for every application.

The point is that most of the time commonly accepted solutions work well. We don't have to reinvent the wheel but should be able to create elegant solutions to new problems.

Important:

Ability of architecting a good architecture for your app is the essential difference between an expert developer and the others. This kind of questions are very general and it has many different approaches.

I have provided an answer with a very concrete structure and it is well enough for you to prove your advanced skills.

115. What are SOLID principles? Can you given an example of each in Kotlin?

SOLID is an acronym for five design principles. These are:

Single Responsibility Principle: A class or method should have only one responsibility.

Open-Closed Principle: A class should be open to be extended, closed to be changed.

Liskov's Substitution Principle: Derived classes must be substitutable for their base classes.

Interface Segregation: Make fine grained interfaces that are client specific.

Dependency Inversion: Depend on abstractions not on concretions.

When we bring the first letters together, it makes: **SOLID** . Let me explain them one by one.

Single Responsibility Principle

Single Responsibility Principle (SRP) states that a class or a method should only be doing one thing and shouldn't be any doing anything related. A class should have only one reason to change. If a class or a method is doing more than one thing, then it must be splitted.

For instance suppose that we have a `Person` class to keep user info such as name and email. Since `Person` is just a model class, it can't have logic of

email validation. Let's see a badly written `Person` class:

```
class Person( val email : String){

  init {
    validateEmail( email )
  }

  fun validateEmail(email: String){
    // Email validation code
  }

}
```

Here we see that Person class has a function `validateEmail( email )` to validate emails. Since Person's only responsibility is to keep person's data, it can not have a function to validate it. Because it is yet another responsibility. It violates SRP.

We need to split it and have 2 classes for two responsibilities: one for holding email data, one for validating it. Let's name second class as .

```
class Person( val email : String){
  init {
    EmailValidator.validateEmail( email )
  }

  // We have removed validateEmail() function
}
// We have created this class for email validation
class EmailValidator{
  companion object {
    fun validateEmail(email: String){
      // Email validation code
    }
  }
}
```

Let's pass to the most famous one.

Open Closed Principle

What it means though is that we should strive to **write code that doesn't have to be changed every time the requirements change** . How we do that can differ a bit depending on the context, such as our programming language. When using Kotlin or some other statically typed language the solution often involves inheritance and polymorphism .

Let's assume we are writing a class for calculating area of rectangle shapes. Here we have a `Rectangle` class and an `AreaCalculator` class that sums area of incoming rectangles with `calculateAreas()` .

```
class Rectangle( val width : Double, val height : Double) {
  fun area() = width * height
}
class AreaCalculator{
  fun calculateAreas(shapes: Array<Rectangle>): Double{
    var totalArea = 0.0
    for (rectangle in shapes){
      totalArea += rectangle.area()
    }
    return totalArea
  }
}
```

At first glance everything seems fine but when we are requested to also calculate area of triangles with rectangles things get messy.

An inexperienced developer would go to `calculateAreas(shapes: Array<Rectangle>)` method and add some code to calculate areas of triangles using if block. That means you have changed the code. But you forgot, our code should be closed to be changed, open to be changed.

Let's see it on codes:

```
class AreaCalculator{
```

```
  fun calculateAreas(shapes: Array<Rectangle>,
triangles: Array<Triangle>): Double{
    var totalArea = 0.0
    for (rectangle in shapes){
      totalArea += rectangle.area()
    }
    for (triangle in triangles){
      totalArea += triangle.area()
    }

    return totalArea
  }
}
```

Liskov Substitution Principle (LSP)

It means child classes should never break the parent class' type definitions.

As simple as that, a subclass should override the parent class' methods in a way that does not break functionality from a client's point of view. Here is a simple example to demonstrate the concept.

Let's consider an `Animal` parent class.

```
interface Animal {
  fun makeNoise()
}
```

Now let's consider the `Cat` and `Dog` classes which extends Animal.

```
class Dog : Animal {
  override fun makeNoise() {
    println ( "bow wow" )
  }
}
class Cat : Animal {
  override fun makeNoise() {
    println ( "meow meow" )
  }
}
```

Now, wherever in our code we were using Animal class object we must be able to replace it with the `Dog` or `Cat` without exploding our code. What do we mean here is the child class should not implement code such that if it is replaced by the parent class then the application will stop running. For example if the following class is replace by `Animal` then our app will crash.

```
internal class DeafDog : Animal {
  override fun makeNoise() {
    throw RuntimeException( "I can't make
noise" )
  }
}
```

If we take Android example then we should write the custom RecyclerView adapter class in such a way that it still works with RecyclerView. We should not write something which will lead the RecyclerView to misbehave.

Interface Segregation Principle

This principle suggests that “many client specific interfaces are better than one general interface”. This is the first principle which is applied on interface, all the above three principles applies on classes. Let’s take following example to understand this principle.

In other words ake fine grained interfaces that are client specific. The interface-segregation principle (ISP) states that no client should be forced to depend on methods it does not use.

Let me give an example from Android. As you know, the Android View class is the root superclass for all Android views.

```
interface OnClickListener {
  fun onClick(view: View);
```

```
    fun onLongClick(view: View);
    fun onTouch(view: View, event: MotionEvent);
}
```

As you can see, this interface contains three different methods, assuming that we wanna get click from a button:

```
// Violation of Interface segregation principle
val button: Button = findViewById(R.id.button);
button.setOnClickListener( View.OnClickListener
{
    override fun void onClick(view: View) {
        // TODO: do some stuff...
    }
    override fun void onLongClick(view: View) {
        // we don't need to it
    }
    override fun void onTouch(view: View, event:
MotionEvent) {
        // we don't need to it
    }
} );
```

The interface is too fat because it's forcing to implement all the methods, even if it doesn't not need them. Let's trying to fix them using **ISP** :

```
// Fix of Interface Segregation principle
interface OnClickListener {
    fun onClick(v: View)
}
interface OnLongClickListener {
    fun onLongClick(v: View)
}
interface OnTouchListener {
    fun onTouch(v: View, event: MotionEvent)
}
```

Now we can use the interface without implement some messy methods. Because it satisfies **Interface Segregation Principle** .

Dependency Inversion Principle

It means high-level modules should not depend on low-level modules. Both should depend on abstractions.

Let me explain with a simple code:

```
// violation of Dependency inversion principle
// Program.java
class Program {
   fun  work() {
      // ....code
   }
}
// Engineer class
class Engineer ( var program: Program){

   fun manage() {
     program.work()
   }
}
```

This code seems pretty familiar to us because most of the time we write this kind of classes.

The problem with the above code is that it breaks the Dependency Inversion Principle : High-level modules should not depend on low-level modules. Both should depend on abstractions.

We have the `Engineer` class which is a high level class, and the low level class called `Program` . And `Engineer` depends on `Program` .

Solution is to define interfaces for `Program` and let `Engineer` to depend on that interface not concrete class.

```
// Dependency Inversion Principle - Good example
interface IProgram {
   fun work()
}
class Program : IProgram {
   override fun work() {
```

```
    // ....code
  }
}
class SuperProgram : IProgram {
  override fun work() {
    //....code
  }
}
class Engineer( var program : IProgram) {

  fun manage() {
    program .work()
  }
}
```

Here we see how better it is.

Main idea behind design principles is that we make our code open to improvements and lots of developers can work it on at the same time. Robert C. Martin has brought these principles together and made us a big fever. Design principles are more abstract compared to design patterns and must be our guide while making decisions.

Important:

OK, this is pretty long answer but it should be. Because principles and patterns are core topics for expert developers. If you are claiming that you are an expert developer, the first thing that you are going to be asked is a pattern or principle.

This is a very generic and huge topic. But we have summarized it all well with nice explanations and code examples.

We recommend you, again, to read the answer 10 times and then you will be able to explain it easily. You should also try to write code on whiteboard or a piece of paper.

Assume that you are asked a principle or pattern. You have explained its meaning well, then your interviewer is probably going to ask

you how to implement it. So, you should be ready to show some code on whiteboard.

You don't need an extra resource to answer this question. This answer is pretty enough for a hard interview.

116. What is Functional Programming and Reactive Programming?

Functional programming is a programming paradigm where we model everything as a result of a function that avoids changing state and mutating data.

On the other side, **Reactive programming** is the practice of programming with asynchronous data streams or event streams.

An event stream can be anything like keyboard inputs, button taps, gestures, GPS location updates, accelerometer, and iBeacon. You can listen to a stream and react to it accordingly.

Let's compare them on code examples. Functional programming is quite straightforward.

```
val numbers = arrayOf ( 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9 )
val numbersLessThanFive = numbers. filter { it <
5 }
```

But when it comes to Reactive programming, we program with event streams. For instance:

```
var twoDimensionalArray = arrayOf ( arrayOf ( 1 ,
2 ), arrayOf ( 3 , 4 ), arrayOf ( 5 , 6 ))
```

```
val flatArray = twoDimensionalArray.flatMap {
  it . map { x * 2 }
}
print (flatArray)
Output : [ 2 , 4 , 6 , 8 , 10 , 12 ]
```

In Android, we have RxAndroid library for reactive programming. It is flexible to use Rx with Retrofit.

CHAPTER 16

CLEAN CODE, CODING CONVENTIONS, BEST PRACTICES

117. What is clean code? How do you write clean code?

Actually, Clean Coding is famous thanks to Uncle Bob. He has a book with same name: Clean Code. Most of the developers have learned clean coding from him.

Clean code is simply writing simple, human readable and good code. Uncle Bob says " bad code smells. ".

I generally pay attention to such things:

> Methods should be small. Uncle Bob says a method should be 3-4 lines but i think that in real world 10-15 lines is enough. If a method is more than 10-15 lines, i think of how to split it.
>
> Naming conventions is very important. Method names, variable names and class names should be understandable. It must propose its intention. It doesn't matter how long they are, but must be named comprehensively.
>
> Indentation is yet another topic. But i don't have to pay effort for this. Just Cmd+Shift+F and Android Studio formats code for me.
>
> Code must be testable. This is a large topic, but in short, try to take things as parameters. Use Dependency Injection if possible. Reduce dependencies. Stay away from Singletons and static variables.
>
> Methods should do only one job. Actually this is Single Responsibility Principle. Code also should satisfy SOLID Design Principles.
>
> Always refactor. I generally review codes that i have written previously and think of how to improve it.
> Don't repeat yourself. This is a famous principle. Repeating codes should be one method and used when required.

Be careful when using external libraries. Just use the libraries that have proven themself. For small tasks, i try to implement it myself rather than using a library.

Restrict method parameter count to three. If more, i write a request class for that method.

Actually this is a huge topic but most of the time, i care about such things. Of course this list can be extended.

118. Which coding conventions you follow when developing an Android app?

Google provides lots of conventions for us. Let me mention just a few of them.

Don't ignore exceptions. We must handle every exception in our code in some principled way.

Don't catch generic exception.

We don't use finalizers. There are no guarantees as to when a finalizer will be called.

Fields should be defined at the top of the file and they should follow the naming rules listed below.

Private, non-static field names start with m.
Private, static field names start with s.
Other fields start with a lowercase letter.
Static final fields (constants) are ALL_CAPS_WITH_UNDERSCORES.

Class member ordering should use the following order:

Constants
Fields
Constructors
Override methods and callbacks (public or private)
Public methods
Private methods
Inner classes or interfaces

When programming for Android, it is quite common to define methods that take a Context. If we are writing a method like this, then the Context must be the first parameter.

Callback interfaces that should always be the last parameter of s method.

When an Activity or Fragment expects arguments, it should provide a public static method that facilitates the creation of the relevant Intent or Fragment. This method is called as `newIntent()` method.

Resource IDs and names are written in **lowercase_underscore** .

Important:

Google provides lots of coding conventions to Android developers. We are supposed to follow these rules.

You can see the full list from this url: https://source.android.com/setup/contribute/code-style

119. What are your best practices as an Android developer?

I have created a lot of Android projects and crafted robust architectures. So far i have learned some best practices. Let me summarize a few of them.

I always try to follow recommended architecture. Google recommends us to split Data and Presentation layers. I also add a Domain layer.

I always use Fragments. I never design my screens on activities.

I always try to satisfy SOLID design principles and design patterns.

I always create a TAG string and a `newIntent()` or `newInstance()` method for every Activity and Fragment.

Unit testing is a must for me. I always include automated tests. I know TDD is hard, so most of the time i don't do TDD but i always add tests for some edge cases and essential scenarios.

I use Proguard in my release version. This will remove all unused code, which will reduce APK size.

I always run my code on different emulators for testing different Android versions and different screen sizes. For instance a 4 inch

phone, a normal phone, 7 inches, 9 inches and 10 inches tablets.

I never hard-code strings. Strings.xml file for this. It also provides i18n for different languages.

I use shapes, selectors and webp format instead of images as much as possible. This further reduces APK size.

I avoid deep levels in layouts. A deep hierarchy of views makes our UI slow, not to mention making it harder to manage your layouts. I use Constraint Layout for this.

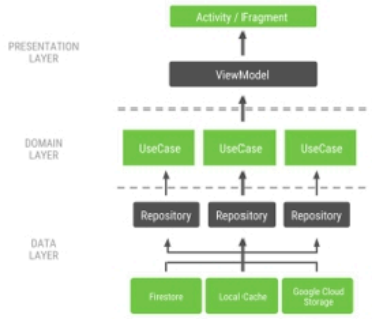
I perform file, network and database operations off the UI thread. Also every heavy calculation should be done off the UI thread.

120. What is Clean Architecture?

Clean Architecture has been proposed by Uncle Bob. The app is divided into a three layer structure:

Presentation layer (Views and ViewModels)
Domain layer (use cases)
Data layer (repositories, user manager)

The presentation layer cannot talk to the data layer directly. A ViewModel can only get to a repository through one or more use cases. This limitation ensures independence and testability. It also brings a nice opportunity to jump to a background thread: all use cases are executed in the background guaranteeing that no data access happens on the UI thread.



Presentation layer: Views + ViewModels + Data Binding
ViewModels provide data to the views via LiveData. The actual UI calls are done with Data Binding, relieving the activities and fragments from boilerplate.

We deal with events using an event wrapper, modeled as part of the UI's state.

Domain layer : UseCases
The domain layer revolves around the UseCase class, which went through a lot of iterations. In order to avoid callback hell we decided to use LiveData to expose the results of the UseCases.

By default use cases execute on a **DefaultScheduler** (a Kotlin object) which can later be modified from tests to run synchronously. We found this easier than dealing with a custom Dagger graph to inject a synchronous scheduler.

Data layer
The app dealt with 3 types of data, considering how often they change:

> Static data that never changes: map, agenda, etc.
> Data that changes 0–10 times per day: schedule data (sessions, speakers, tags, etc.)
> Data that changes constantly even without user interaction: reservations and session starring

121. You are supposed to review someone's code. How do you judge it?

The first thing i care about is commit message. We developers use source control for collaborating. We use Git for it. So every commit on Git must be clear enough to expose its intent. Commit messages must be clean and neat.

Secondly, i pay attention to version code, fix numbers, Git tag etc... Such things are also important. If there is no problem, then i look at the code.

If a commit is too large, i deny it. Commits must be small. Google recommends maximum of 30 lines of code per commit but we can extend it to 50-60 lines at our company.

Looking at code, i ask these questions to myself:

> Is code formatted?
> Variable and method names are clear enough?
> SOLID design principles are violated?
> Is there a better way of implementing this?
> Is the code readable, short and clean?

I judge the code against to these criterion.

Lastly, is unit tests are included? I definitely reject commits without unit tests.

CHAPTER 17

ARCHITECTURE COMPONENTS

121. What are Architecture Components?

Android architecture components are a collection of libraries that help you design robust, testable, and maintainable apps. They are a part of Jetpack.

It provides us classes for managing our UI component lifecycle and handling data persistence. Basically, it provides four components.

Lifecycle: We use Lifecycle to manage our app's lifecycle with ease. New lifecycle-aware components help us manage our activity and fragment lifecycles. Survive configuration changes, avoid memory leaks and easily load data into our UI.

LiveData: We use LiveData to build data objects that notify views when the underlying database changes.

ViewModel: ViewModel Stores UI-related data that isn't destroyed on app rotations.

Room: Room is an a SQLite object mapping library. We use it to avoid boilerplate code and easily convert SQLite table data to Java objects. Room provides compile time checks of SQLite statements and can return RxJava, Flowable and LiveData observables.

122. How does ViewModel survives configuration changes?

`ViewModel` objects are scoped to the Lifecycle passed to the `ViewModelProvider` when getting the `ViewModel`. The `ViewModel` remains in memory until the Lifecycle it's scoped to goes away permanently: in the case of an activity, when it finishes, while in the case of a fragment, when it's detached.

We should never use constructor to get an instance of a `ViewModel`. Because that defeats the `ViewModel`'s purpose of surviving configuration changes. Android provides us ViewModelProvider factory class for instantiating a ViewModel.

`ViewModelProvider` uses a HashMap in its internal implementation and provides us same instance of `ViewModel` everytime.

A typical example would be:

```
class MyActivity : AppCompatActivity() {
  override fun onCreate(savedInstanceState:
Bundle?) {
    super .onCreate(savedInstanceState)
```

```
    var model =
ViewModelProviders.of( this ).get(MyView-
Model:: class . java )
    model.getUsers().observe( this , users -> {
      // update UI
    } )
  }
}
```

123. What is advantage of LiveData?

LiveData is an observable data holder class. Unlike a regular observable, LiveData is lifecycle-aware, meaning it respects the lifecycle of other app components, such as activities, fragments, or services. This awareness ensures LiveData only updates app component observers that are in an active lifecycle state.

LiveData considers an observer, which is represented by the Observer class, to be in an active state if its lifecycle is in the **STARTED** or **RESUMED** state. LiveData only notifies active observers about updates. Inactive observers registered to watch LiveData objects aren't notified about changes.

That is the advantage of LiveData over a regular observable.

124. Can you explain relationship between LifeCyle Owner and LifeCycle Observer?

LifecycleOwner is a single method interface that denotes that the class has a Lifecycle. It has one method, *getLifecycle()* , which must be implemented by the class.

This interface abstracts the ownership of a Lifecycle from individual classes, such as Fragment and AppCompatActivity , and allows writing components that work with them. Any custom application class can implement the LifecycleOwner interface.

Components that implement **LifecycleObserver** work seamlessly with components that implement **LifecycleOwner** because an owner can provide a lifecycle, which an observer can register to watch.

125. What do you know about ROOM?

Room is an a SQLite object mapping library . The Room persistence library provides an abstraction layer over SQLite to allow for more robust database access while harnessing the full power of SQLite.

The library helps us create a cache of our app's data on a device that's running our app. This cache, which serves as our app's single source of truth, allows users to view a consistent copy of key information within our app, regardless of whether users have an internet connection.

We should use it to avoid boilerplate code and easily convert SQLite table data to Java objects. Room provides compile time checks of SQLite statements and can return RxJava, Flowable and LiveData observables.

CHAPTER 18

THIRD PARTY LIBRARIES

126. What is RxAndroid? How can we benefit from it?

RxAndroid is a reactive programming library for Android. **Reactive Programming** is basically event-based asynchronous programming. Everything we see is an asynchronous data stream, which can be observed and an action will be taken place when it emits values.

We can create data stream out of anything; variable changes, click events, http calls, data storage and errors. When it says asynchronous, that means every code module runs on its own thread thus executing multiple code blocks simultaneously.

A typical example of RxAndroid would be:

```
myObservable // observable will be subscribed on i/o thread
    .subscribeOn(Schedulers.io())
    .observeOn(AndroidSchedulers.mainThread())
    .map { /* this will be called on main thread... */ }
    .doOnNext { /* ...and everything below until next observeOn */ }
    .observeOn(Schedulers.io())
    .subscribe { /* this will be called on i/o thread */ }
```

Rx also provide RxKotlin, which is a wrapper of RxJava. We can safely use it.

127. What is Dependency Injection? How do you implement it in your project?

Dependency injection is a very powerful technique, where you relay the task of providing object with its' dependencies on instances of other objects to a separate class.

This allows for fewer constructors, setters, factories and builders as all those functions are taken care of by the DI framework that we use.

The pattern seeks to establish a level of abstraction via a public interface and to remove dependencies on components by supplying a

'plugin' architecture. This means that the individual components are tied together by the architecture rather than being linked together themselves. The responsibility for object creation and linking is removed from the objects themselves and moved to a factory.

Another example of DI usage is unit-testing - it allows to conveniently inject all needed dependencies and keep the amount of written code at a lower level.

In Android, we use **Dagger** library for dependency injection. Dagger is a fully static, compile-time dependency injection framework for both Java and Android. It is an adaptation of an earlier version created by Square and now maintained by Google.

Dagger 2 uses annotation processing not reflection. That is why it is so fast and successful. In order to use it, we add the dependency to our Gradle file and write a component interface and a module class for ith. Then we use it with `@Inject` annotation and it provides us the object we want.

For instance;

```
class MyActivity : AppCompatActivity() {
  @Inject lateinit var navigator : Navigator
  @Inject lateinit var moviesAdapter :
MoviesAdapter

}
```

128. What is advantage of Dagger 2 against other DI libraries?

Dagger 2 works on **Annotation processor** . So all the code generation is traceable at the compile time. Hence we have no performance overhead and its errors are highly traceable.

That is the main advantage of Dagger 2 over other dependency injection frameworks. That is why it's so fast and successful.

Annotation processors are the code generators that eliminate the boilerplate code, by creating code for us during the compile time. Since it's compile time, there is no overhead in the performance.

Dagger provides three different usage of Inject annotations: **Field Injection** , **Constructor Injection** , **Method Injection** .

```
class MyActivity {
   /**
   * Explaining different usage of Inject annotations
of dagger
   **/
  //Feild injection
   @Inject Allies allies;
   //Constructor injection
   @Inject
   public MyActivity()
   {
      //do something..
   }
   //Method injection
   @Inject
   private void someMethod()
   {
      //do something..
   }
}
```

129. How does ButterKnife works? Can you explain its working mechanism?

ButterKnife works thanks to **Annotation Processing** . Annotation processing is a tool build in *javac* for scanning and processing annotations at compile time.

When we compile our Android project ButterKnife Annotations Processor *process()* method is executed doing the following:

First, it scans all java classes looking for ButterKnife annotations: *@InjectView* , *@OnClick* , etc.

When it find a class with any of these annotations it creates a file called: *<className>$$ViewInjector.java*

This new *ViewInjector* class contains all the necessary methods to handle annotation logic: *findViewById()* , *view.setOnClickListener()* , etc.

Finally, during execution, when we call *ButterKnife.inject(this)* each ViewInjector *inject()* method is called.

130. What are commonly used libraries in a typical Android app?

I pay attention to use commonly used libraries because they give us power and flexibility. To name just a few of them:

RxJava: reactive programming for Java and Android. Also supports Kotlin.

Dagger2: The most popular Dependency Injection framework. Maintained by Google.

jUnit 4: Official test framework of Java and Android.

Mockito: Mockito allows you to create and configure mock objects. The most popular mock framework which can be used in conjunction with JUnit.

Robolectric: A unit test framework that defangs the Android SDK jar so we can test-drive the development of our Android app.

Espresso: A library to write concise, beautiful, and reliable Android UI tests.

Retrofit: A must use tool for connecting to a web service.

Fresco: Fresco is a powerful system for displaying images in Android applications.
Fabric - Crashlytics: The most powerful, yet lightest weight crash reporting solution.

Google Analytics: Helps us to measure what is happening in our app.

Architecture Components: Helps us design more robust, testable and maintainable apps.

Glide: For downloading and caching images.

EventBus: Simplifies communication between Activities, Fragments, Threads, Services, etc. Less code, better quality.

LeakCanary: We use it to find memory leaks.

These technologies are kinda toolbelt for a well grounded Android developer. I efort to use these latest technologies effectively.

131. What do you know about Version Control? Have you ever used Git?

Version control systems are a category of software tools that help a software team manage changes to source code over time.

Version control software keeps track of every modification to the code in a special kind of database. If a mistake is made, developers can turn back the clock and compare earlier versions of the code to help fix the mistake while minimizing disruption to all team members.

The most popular version control system is Git. I have been using it for years.

Git is a distributed version control system. Every dev has a working copy of the code and full change history on their local machine, by Linus Torvalds.

Using Git, we are able to collaborate on the same project, version our code, commit and push to repository.

Let me briefly explain its working mechanism...

When working with Git, we have a local repository and a remote repository. We pull the source code from remote repository, make changes and commit code to local repository then we push to the remote repository. Then other teammates

can pull the changes that we made on project to their locale machines.

Git provides us ability of creating branches. So that we can work on different versions of the project without mixing up the codes.

Let me show you most used Git commands.

To create an empty git repo or reinitialize an existing one

```
$ git init
```

To c lone the foo repo into a new directory called foo:

```
$ git clone https://github.com/<username>/foo.git foo
```

To see a list of the repositories (remotes) whose branches we track:

```
$ git remote -v
```

To create and checkout a new branch:

```
$ git checkout -b <new_branch_name>
```

To see what files have changed:

```
$ git status
```

To commit a change:

```
$ git commit -m "Updated README"
```

To push a local branch for the first time:

```
$ git push --set-upstream origin <branch>
$ git push
```

132. What is Continuous Integration?

Continuous Integration (CI) is a development practice that requires developers to integrate code into a shared repository several times a day. Each check-in is then verified by an automated build, allowing teams to detect problems early.

By integrating regularly, we can detect errors quickly, and locate them more easily.

We use **Jenkins** as our CI server. We have a bunch of unit and integration tests. Whenever we submit a build request, Jenkins runs all the automated tests and emails us a report.

Martin fowler says that “Continuous Integration doesn’t get rid of bugs, but it does make them dramatically easier to find and remove.”

CHAPTER 19

UNIT TESTS & INSTRUMENTATION TESTS

133. Uncle Bob has a famous acronym for tests: FIRST. Do you know what does it mean?

First is an acronym. It stands for Fast, Independent, Repeatable, Self-Validating and Timely.

Let me explain them one by one.

Fast
Unit tests should be fast otherwise they will slow down our development/deployment time and will take longer time to pass or fail.

Independent
Never ever write tests which depend on other test cases. We should be able to run any one test at any time, in any order.

By making independent tests, it's easy to keep our tests focused only on a small amount of behavior.

Repeatable
A repeatable test is one that produces the same results each time you run it. To accomplish

repeatable tests, we must isolate them from anything in the external environment not under our direct control.

Self-validating
Tests must be self-validating means – each test must be able to determine that the output is expected or not. It must determine it is failed or pass. There must be no manual interpretation of results.

Timely
Practically, we can write unit tests at any time. We can wait upto code is production ready or we're better off focusing on writing unit tests in a timely fashion.

134. What is difference between Unit Test and Instrumentation Test?

Unit tests run tests without a device or an emulator attached. Thay are referred to as "local tests" or "local unit tests".

Instrumentation tests run on a device or an emulator. In the background, our app will be installed and then a testing app will also be installed which will control our app, launching it and running UI tests as needed.

Unit tests cannot test the UI for our app without mocking objects such as an Activity.

Instrumentation tests can be used to test none UI logic as well. They are especially useful when we need to test code that has a dependency on a context.

JUnit is the only thing we need for unit testing but we need Espresso for instrumented tests.

Lastly, unit test focuses on objects behaviour but instrumented tests are generally for testing integration between components such as Activity or Fragments etc...

135. What is TDD (Test Driven Development) ?

Test-driven development (TDD) (Beck 2003; Astels 2003), is an approach to development which you write a test before you write just enough production code to fulfill that test and refactoring.

TDD has a very short development cycle: first the developer writes a failing automated test case that defines a desired improvement or new function, then produces the minimum amount of code to pass that test, and finally refactors the new code to acceptable standards.

The following sequence of steps is generally followed:

Add a test
Run all tests and see if the new one fails
Write some code
Run tests
Refactor code
Repeat

TDD forces us to think through our requirements or design before we write our functional code. TDD is both an important agile require-

ment and agile design technique. The goal of TDD is to write clean code that works.

Actually TDD is very hard to follow. Recently there is a rumor that TDD is dying because almost no one doing it. You need to practice well to master TDD. At that point you are gonna benefit it too much but until that point you are going to suffer.

I personally support TDD but i think we need much more time, a better deadline for project delivery if we are following TDD.

136. How can you test a timeout? Write a simple unit test method for it.

There are couple solutions how we can test a timeout.

The first solution is to use `timeout` parameter in `@Test` annotation. The time is in milliseconds.

```
@Test ( timeout= 100 )
public void myTestFunction() { ... }
```

The other solution is to use `@Rule` annotation. This is really handy if we want to setup timeout for the whole test case for every test in the file.

```
class MyTests {
  @Rule
   public Timeout globalTimeout = Timeout.sec-
onds( 10 );
   @Test
   public void test1() throws Exception {
```

```
    ...
  }
  @Test
  public void test2() throws Exception {
    ...
  }
}
```

If the timeout is reached, it will throw `TimeoutException` .

137. How can you test an Exception? Write a simple unit test method for it.

Actually i know three different ways of this. The first is to use `@Test ( expected )` annotation.

```
@Test ( expected = IndexOutOfBoundsException:: class )
fun empty() {
  ArrayList<Any>()[ 0 ]
}
```

That is good for short test methods. The above test will pass if any code in the method throws *IndexOutOfBoundsException* .

Secondly, for longer test methods it's recommended to use the *ExpectedException* .

```
@Rule
var thrown = ExpectedException.none()
@Test
@Throws (IndexOutOfBoundsException:: class )
```

```
fun shouldTestExceptionMessage() {
   val list = ArrayList<Any>()
   thrown .expect(IndexOutOfBoundsException::
class . java )
   thrown .expectMessage( "Index: 0, Size: 0" )
  list[ 0 ] // execution will never get past this line
}
```

This rule lets you indicate not only what exception we are expecting, but also the exception message we are expecting.

Lastly, we may use **Try/Catch** idiom for testing purposes. A simple explanation would be:

```
@Test
fun testExceptionMessage() {
   try {
     ArrayList<Any>()[ 0 ]
     fail( "Expected an IndexOutOfBoundsExcep-
tion to be thrown" )
  } catch (anIndexOutOfBoundsException: Index-
OutOfBoundsException) {
     assertThat(anIndexOutOfBoundsException.
message , `is`( "Index: 0, Size: 0" ))
  }
}
```

138. We want to test business logic of our app but it's implemented in an activity or fragment. How do you solve it?

First of all we need to remove business logic from our UI components. There shouldn't be any logic in an Activity or Fragment.

I hold UI logic and UI data in **ViewModels** . So we may write a ViewModel class for every Fragment in our app and move UI logic to there.

Additionally, we need an extra layer for our business logic. This layer may be called as domain layer.

Clean Architecture recommends us to divide our project into three layers : Presentation, Domain and Data.

We keep UI related classes in **Presentation** . These are Activity, Fragment or ViewModel. There shouldn't be any business logic here. Only UI related things are allowed.

We use data related things in **Data** layer. Retrofit, ROOM, repositories etc...

And we have **Domain** layer for business logic. We are not allowed to reach other layers from this layer. It's only pure Kotlin or Java.

Since we have sealed our business logic in domain layer, we can easily test our business logic simply using only JUnit. We don't need any real device or emulator.

That is also satisfies **Separation of Concerns** principle.

139. How can you effectively deal with legacy code?

Michael Feathers has written a famous book for this : **Working Effectively with Legacy Code** . We all need to read this.

He recommends a few steps. Let me summarize them one by one.

1. We should write characterization tests
Our goal isn't to figure out what the system should do but rather what it actually does, hence the name characterization. We just there to observe and categorize.

2. We should bring in tooling to help our testing and characterization efforts
Writing characterization tests is great in principle. And it can work well in practice when a couple of things are true:

> We have a relatively isolated bit of functionality, like the aforementioned, hypothetical `Add()` method.
> The scope of our change is relatively limited and narrow.

3. We should keep changes to a minimum and boy-scout as you go
We should avoid changing anything we don't need to, but when we have to change something, we should try also to improve it a little. After all, we're probably not done with this codebase after one single change.

4. We should work gradually toward modularity and better design
Well, because if we make the codebase modular enough, we can start to migrate parts of it to newer platforms, or modernize it with language/framework upgrades.

140. What is advantage of automated testing? Why do we write it?

Automated testing ensures that you haven't broken any functionality previously implemented. We constantly keep adding new code or making changes to our codebase. Hence we should be sure that we haven't broken any functionality that is currently working.
Automated tests gives us confidence.

Automated testing cuts down the time to run repetitive tests from days to just a few hours. After all less time spent equals smaller development bill to foot.

Additionally, we don't need to keep a dedicated manual testing team to handle QA. One automated testing engineer can have us covered at all times.

141. How do you structure your unit test methods?

I generally use **Given-When-Then** structure for my tests.

We define variables in **Given** section.

When section is where the action happens. Methods to be tested are called here.
Then section is where we check result and decide pass or fail.

This approach is recommended in **Clean Code** book of Uncle Bob. A simple example would be as follows:

```
@Test
fun getName_returnsName() {
  // GIVEN
  val expectedName = "linda"
  testSubject = Customer(expectedName)
  // WHEN
  val actualName = testSubject.getName()
  // THEN
  assertTrue(expectedName == actualName)
}
```